

Sequential Workflows in LangGraph | Agentic AI using LangGraph

1. Introduction

- A **sequential workflow** executes tasks in a fixed order.
- Each step runs after the previous step finishes.
- Basic structure:

Step 1 → Step 2 → Step 3 → Step 4

- This is one of the simplest workflow patterns in LangGraph.

2. Installation

- Install LangGraph and the required dependencies in your Python environment.
- Typically, you also need an **LLM provider package** if the workflow will use an LLM.
- Keep dependencies inside a virtual environment to avoid conflicts.

3. First Workflow

A basic LangGraph workflow consists of:

- **State** — Data shared between workflow steps.
- **Nodes** — Functions that perform tasks.
- **Edges** — Connections that determine execution order.
- **Graph** — The complete workflow.

Example:

Input → Node A → Node B → Node C → Output

Each node receives the current state, performs its task, and can update the state.

The graph is then **compiled** and executed with an initial state.

4. LLM Workflow

LangGraph can use LLMs inside workflow nodes.

Example:

User Input → LLM Node → Processing Node → Final Output

Each node can perform a specific LLM-related task.

For example:

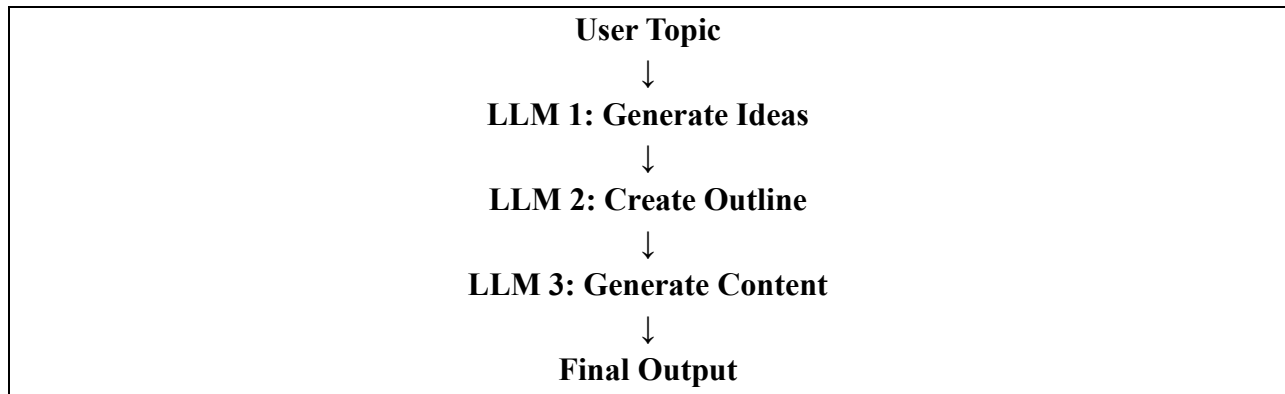
1. Receive a user's topic.
2. Ask the LLM to generate an explanation.
3. Process or transform the generated content.
4. Produce the final result.

The important idea is that **LLM calls become individual steps inside the graph**.

5. Prompt Chaining

Prompt chaining connects multiple LLM calls sequentially.

Example:



The output from one prompt becomes the input for the next step.

This is useful when one large prompt would be difficult to manage.

Key benefit: A complex task is divided into **small, focused LLM operations**.

6. Outro

- Sequential workflows are useful when tasks have a **clear and fixed order**.
- The core LangGraph structure is:

State + Nodes + Edges → Graph → Execution

- When workflows become more complex, LangGraph can extend this basic pattern with **routing, parallel execution, loops, and conditional logic**.